

PORT SCANNER PROJECT REPORT

Submitted By

Name: Sanovar Sheikh

Course: BCA (2nd Year)

College: G.H Rasoni Nagpur

Submitted To : C14-CodeTech

Project Title

Metadata Extractor using Kali Linux

Academic Year

2025 – 2026

Introduction

A port scanner is a software tool used to scan a network or system to identify open ports. Each port on a system is associated with a specific service such as HTTP (port 80), FTP (port 21), and SSH (port 22). Open ports indicate that a service is running and accepting connections.

Port scanning is an important technique used by network administrators and cybersecurity professionals to monitor network security, detect vulnerabilities, and prevent unauthorized access. This project demonstrates how a simple port scanner can be built using Python in Kali Linux.

Objective

1. To develop a basic port scanner using Python programming language.
2. To understand how network ports and IP addresses function.
3. To identify open ports and services running on a target system.
4. To learn the use of Python socket library for network communication.
5. To gain practical knowledge of cybersecurity and ethical hacking concepts.
6. To analyze network vulnerabilities by detecting open ports.
7. To build a simple and efficient port scanning tool.

Tools & Technologies Used

- Kali Linux (Operating System)
- Python 3 (Programming Language)
- Socket Library (for network communication)

Working Principle

The port scanner works on the concept of client-server communication.

- The program sends a connection request to a specific port on the target IP address.
- If the port responds successfully, it means the port is open and a service is running.
- If the port does not respond or connection fails, it is considered closed.

The "socket.connect_ex()" function is used because it returns an error code instead of stopping the program, making scanning faster and efficient.

Steps to Create the Project

1. Open Kali Linux terminal
2. Create a Python file:
 `nano portscanner.py`
3. Write Python code using socket library
4. Save the file (CTRL + O, ENTER, CTRL + X)
5. Run the program:
 `python3 portscanner.py`
6. Enter target IP address (e.g., 127.0.0.1 or any website IP)
7. The program scans ports and displays open ones

Sample Code

```
import socket

target = input("Enter target IP: ")

print("Scanning target:", target)

for port in range(1, 101):
    s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    s.settimeout(0.5)

    result = s.connect_ex((target, port))

    if result == 0:
        print(f"Port {port} is OPEN")
    else:
        pass

s.close()
```

Output

Example output:

```
Scanning target: 127.0.0.1
Port 22 is OPEN
Port 80 is OPEN
```

Applications

- Network security analysis
- Detecting open ports and services
- Ethical hacking and penetration testing
- System and server monitoring

Advantages

- Easy to develop and understand
- Helps in identifying security weaknesses
- Useful for beginners in cybersecurity
- Can be modified for advanced scanning

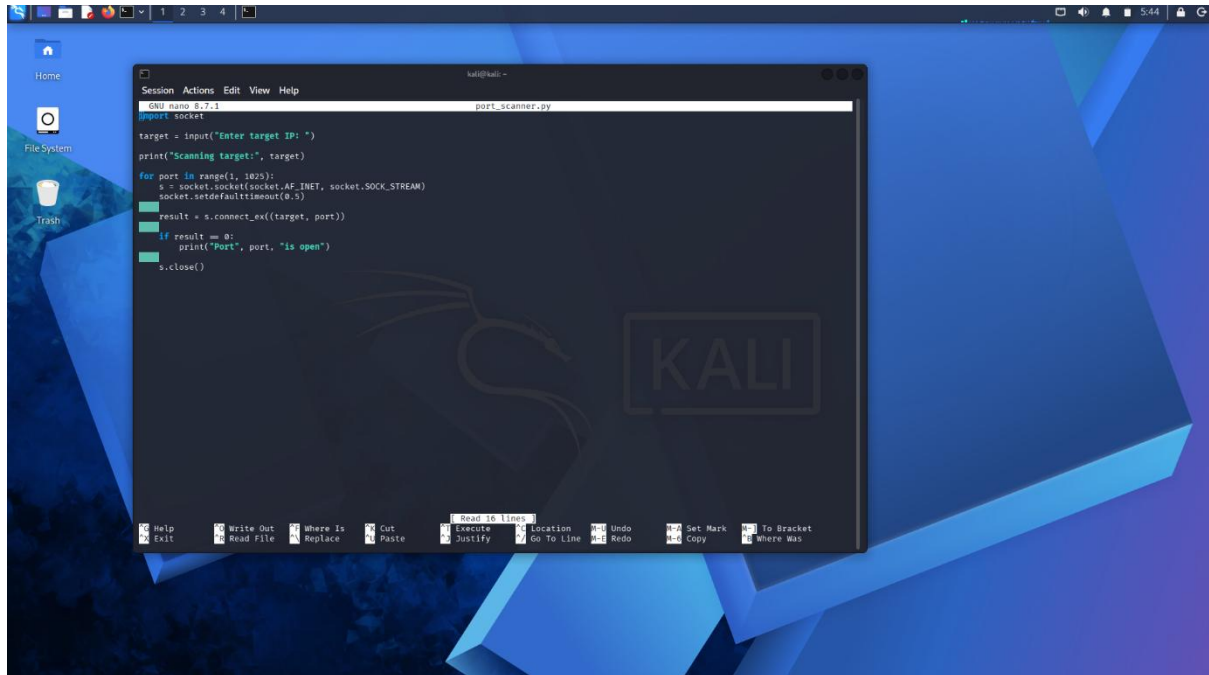
Limitations

- Slow scanning for large port ranges
- May be blocked by firewalls or antivirus
- Does not provide detailed service information
- Basic functionality compared to professional tools

Conclusion

The Port Scanner project successfully demonstrates how to scan and detect open ports on a system using Python. It helps in understanding basic networking concepts and plays an important role in cybersecurity practices. This project serves as a foundation for building more advanced network security tool

Screen Shot Of Code



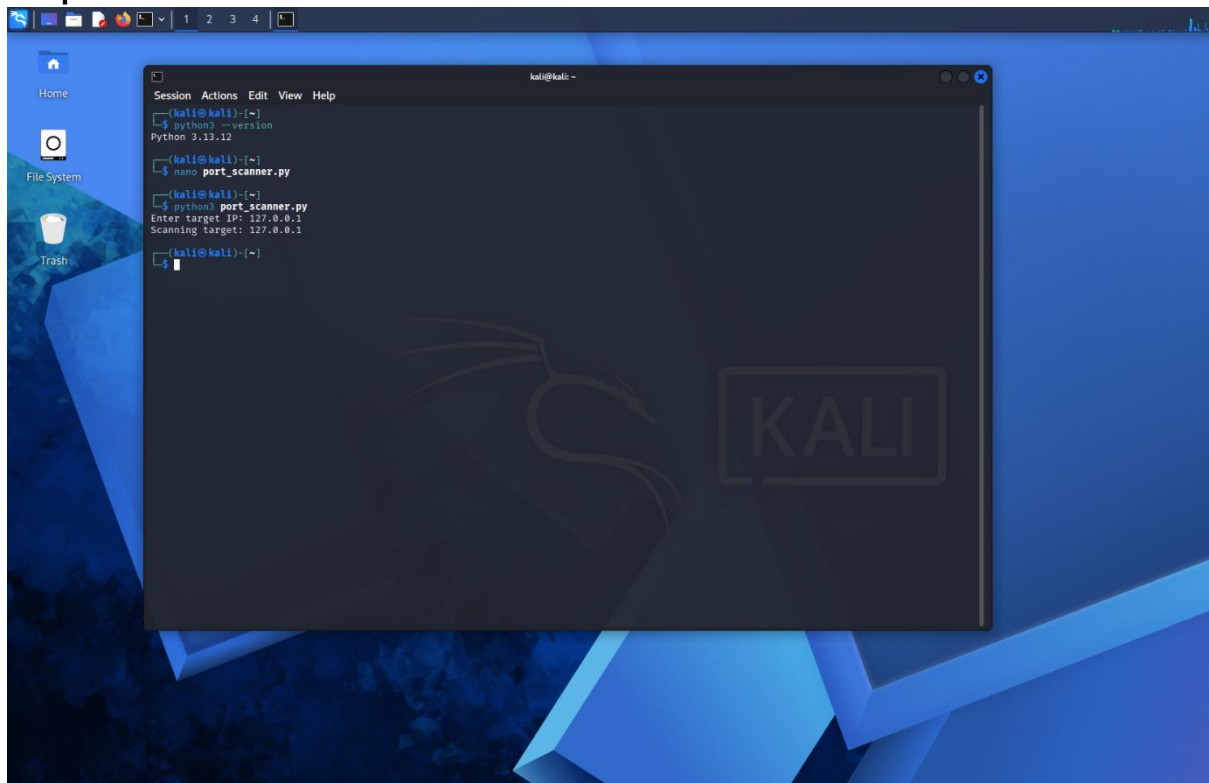
The screenshot shows a Kali Linux desktop environment with a blue and purple geometric background. A nano editor window is open, displaying a Python script named `port_scanner.py`. The script is designed to scan a target IP for open ports. The desktop includes icons for Home, File System, and Trash. The nano editor's menu bar includes Session, Actions, Edit, View, and Help. The script code is as follows:

```
#!/usr/bin/perl
import socket

target = input("Enter target IP: ")
print("Scanning target:", target)

for port in range(1, 1025):
    s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    socket.setdefaulttimeout(0.5)
    result = s.connect_ex((target, port))
    if result == 0:
        print("Port", port, "is open")
    s.close()
```

Output Screenshot



The screenshot shows the same Kali Linux desktop environment. A terminal window is open, showing the execution of the port scanner script. The user has entered the target IP `127.0.0.1`. The script has scanned the target and found no open ports. The terminal output is as follows:

```
(kali@kali)-[~]
└─$ python3 --version
Python 3.13.12

(kali@kali)-[~]
└─$ nano port_scanner.py

(kali@kali)-[~]
└─$ python3 port_scanner.py
Enter target IP: 127.0.0.1
Scanning target: 127.0.0.1

(kali@kali)-[~]
└─$
```